TALLER EVALUATIVO DE PROCESAMIENTO DISTRIBUIDO Y BENCHMARKING EMPÍRICO

Renderizado Paralelo de Fractales con Patrón Master-Worker y Bag of Tasks

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS INTELIGENTES

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09810 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	12378 / 12377	**CARÁCTER:**	Taller Evaluativo / Laboratorio Práctico
MODALIDAD:	Equipos de 2 o 3 Estudiantes	**DURACIÓN:**	2 Horas en Clase + Cierre Autónomo en el Día
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	**TEMA CENTRAL:**	Procesamiento Distribuido (MIMD / NORMA)
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____
ESTUDIANTE 3:	_____	**CÓDIGO:**	_____

1 Contexto del Problema y Desafío de Ingeniería

En etapas previas exploramos el middleware ZeroC Ice y los contratos en lenguaje Slice (`.ice`) para comunicar procesos transparentemente sobre la red. Sin embargo, en el diseño de sistemas distribuidos empresariales y de cómputo en la nube, el middleware es únicamente la tubería básica (*plumbing*). El verdadero desafío de ingeniería consiste en responder: **¿Cómo dividimos un problema computacional pesado y cómo calibramos los parámetros de concurrencia para que el clúster realmente acelere el procesamiento?**

En este taller evaluativo implementaremos y analizaremos un clúster de procesamiento distribuido bajo el patrón arquitectónico **Master-Worker** para renderizar el **Conjunto de Mandelbrot**, un fractal cuya función de escape recursiva ($z_{n+1} = z_n^2 + c$) demanda miles de millones de operaciones de punto flotante.

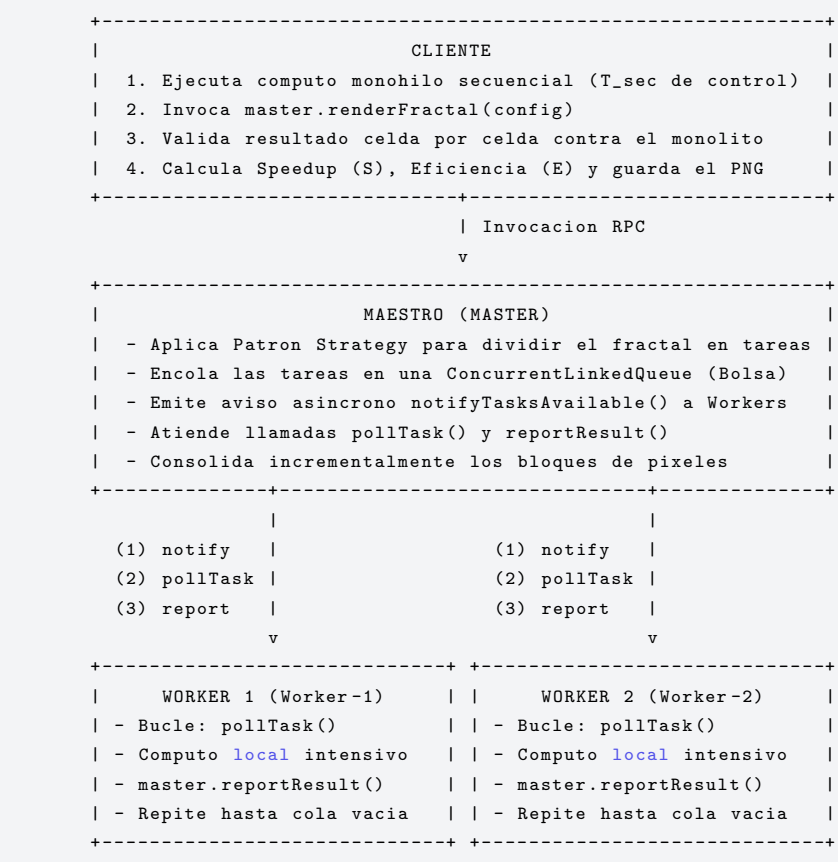
La Trampa del Desbalance de Carga y la Granularidad

El Conjunto de Mandelbrot posee una propiedad fundamental: **su carga de cómputo es heterogénea**. Los píxeles del exterior escapan en 1 o 2 iteraciones (toman nanosegundos), mientras que los píxeles interiores tardan el máximo de iteraciones ($3{,}000$ a $5{,}000$; toman milisegundos). Si un arquitecto divide la imagen en franjas estáticas fijas, el nodo asignado al centro del fractal tardará 20 veces más que los nodos de los extremos, desperdiciando la capacidad del clúster.

2 Arquitectura del Sistema: Extracción Asíncrona (Work-Pulling)

A diferencia de modelos rígidos donde el Maestro mantiene hilos bloqueados esperando a cada Worker, este sistema imple-

menta un modelo desacoplado basado en una **Bolsa de Tareas Concurrente (*Bag of Tasks*)**:



1. **Registro y Espera Pasiva (Standby):** Cada Worker inicia, activa su adaptador con la interfaz `WorkerObserver` y se registra ante el Maestro. Luego queda en reposo sin consumir CPU.
2. **Partición y Notificación Asíncrona:** Al recibir una orden de renderizado, el Maestro aplica el **Patrón Strategy**, genera las tareas, las inserta en su cola y emite una notificación no bloqueante a los Workers.
3. **Extracción Dinámica (*Work-Pulling*):** Cada Worker ejecuta un bucle donde solicita una tarea al Maestro (`pollTask()`). Si hay tarea disponible, la procesa localmente con su pool de hilos, entrega el resultado parcial (`reportResult()`) y solicita inmediatamente la siguiente.
4. **Consolidación en Streaming:** El Maestro ensambla los bloques de píxeles directamente en la matriz final a medida que llegan. Cuando la última tarea es reportada, se completa la promesa y se retorna la imagen al Cliente.

3 Patrón Strategy: Estrategias de Descomposición

El código base desacopla la creación de tareas mediante la interfaz `TaskPartitionStrategy`:

```
package co.icesi.fractal.strategy;

import FractalApp.RenderConfig;
import FractalApp.TaskTile;
import java.util.List;

public interface TaskPartitionStrategy {
    List<TaskTile> partition(RenderConfig config);
    String getStrategyName();
}
```

Se contemplan dos estrategias fundamentales:

1. **Estrategia 1:** `FixedCountPartitionStrategy` **(División en N tareas fijas):** Divide la altura de la imagen en N franjas horizontales contiguas. Cada franja abarca el ancho completo (W) y una altura aproximada de H/N píxeles.
2. **Estrategia 2:** `FixedTileSizePartitionStrategy` **(División por baldosas $S \times S$):** Divide la imagen en una cuadrícula bidimensional de baldosas cuadradas o rectangulares de tamaño fijo (ej. 16×16, 64×64, 128×128).

4 Estructura del Proyecto Gradle Multimódulo

El código se organiza en 5 submódulos en **Gradle 8.6**:

```
fractal-master-worker/
|-- settings.gradle                          # Orquestador multimodulo
|-- build.gradle                             # Configuracion global de Java 17 y Slice plugin
|-- common/                                  # Contrato Slice, MandelbrotCore y Estrategias
|   |-- src/main/slice/FractalCompute.ice    # Definicion IDL (RenderConfig, TaskTile, TileResult)
|   `-- src/main/java/co/icesi/fractal/
|       |-- core/MandelbrotCore.java         # Motor matematico y guardado de imagen PNG
|       `-- strategy/                        # Interfaz y factoria de estrategias
|-- monolith/                                # Linea base secuencial en una sola CPU
|   `-- src/main/java/co/icesi/fractal/monolith/MonolithicMain.java
|-- server/                                  # Servidor Maestro con bolsa de tareas
|   `-- src/main/java/co/icesi/fractal/master/MasterMain.java
|-- worker/                                  # Nodo Trabajador con ciclo de extraccion
|   `-- src/main/java/co/icesi/fractal/worker/WorkerMain.java
`-- client/                                  # Cliente de benchmarking y ejecutor de experimentos
    `-- src/main/java/co/icesi/fractal/client/
        |-- ClientMain.java                  # Verificacion de integridad y calculo de Speedup
        `-- ExperimentRunner.java            # Automatizador de la matriz de pruebas
```

4.1 Comandos de Compilación y Ejecución

1. **Compilar todo el proyecto:**

```
./gradlew build
```

2. **Ejecutar el Monolito (Línea Base Secuencial):**

```
./gradlew :monolith:runMonolith --args="1920 1080 3000 output/mandelbrot_monolith.png"
```

3. **Iniciar el Clúster Distribuido (Terminales Separadas):**

```
# Terminal 1 (Maestro):
./gradlew :server:runMaster --args="10000"

# Terminal 2 (Worker 1 con 4 hilos):
./gradlew :worker:runWorker --args="10001 Worker-1 localhost 10000 4"

# Terminal 3 (Worker 2 con 4 hilos):
./gradlew :worker:runWorker --args="10002 Worker-2 localhost 10000 4"
```

4. **Ejecutar el Cliente de Verificación Única:**

```
# Argumentos: <host> <puerto> <ancho> <alto> <maxIter> <estrategia: 0=franjas, 1=baldosas> <parametro> <salida>
./gradlew :client:runClient --args="localhost 10000 1920 1080 3000 1 64 output/mandelbrot_dist.png"
```

5. **Ejecutar la Matriz Automatizada de Experimentos:**

```
./gradlew :client:runExperiment --args="localhost 10000 1920 1080 2000"
```

5 Protocolo Experimental: Análisis de Decisiones Arquitectónicas

Cada equipo debe ejecutar los siguientes tres experimentos sistemáticos, registrando los tiempos de ejecución y calculando la Aceleración ($S = T_{\text{sec}}/T_{\text{dist}}$) y Eficiencia ($E = S/p \times 100\,\%$):

5.1 Experimento 1: El Dilema de la Granularidad y el Sobrecosto de Red

Fijando $p = 2$ workers activos, renderice una imagen Full HD (1920×1080, $2{,}000$ iteraciones) variando la estrategia y el tamaño de tarea:

Estrategia / Configuración	Parámetro	Tareas Creadas	T_{sec} (ms)	T_{dist} (ms)	Speedup (S)	Eficiencia (E)
FixedCount (Franjas fijas)	$N = 2$	2 franjas				
FixedCount (Franjas fijas)	$N = 4$	4 franjas				
FixedCount (Franjas fijas)	$N = 16$	16 franjas				
FixedTileSize (Grano ultrafino)	Baldosas 16×16	8,160 baldosas				
FixedTileSize (Grano intermedio)	Baldosas 64×64	510 baldosas				
FixedTileSize (Grano grueso)	Baldosas 256×256	40 baldosas				

Tabla 1: Matriz de impacto de la granularidad sobre el rendimiento distribuido.

5.2 Experimento 2: Concurrencia y Saturación del Pool de Hilos en Workers

Fijando la mejor estrategia del Experimento 1 (ej. Baldosas de 64×64), configure un Worker variando la cantidad de hilos de cómputo en su ThreadPool:

- **Configuración A:** 1 hilo (monohilo puro por nodo).
- **Configuración B:** 2 hilos.
- **Configuración C:** $N =$ Cores físicos de la máquina (ej. 4 u 8 hilos).
- **Configuración D:** 64 hilos (sobredimensionamiento extremo).

5.3 Experimento 3: Escalabilidad del Clúster y Ley de Amdahl

Ejecute la configuración óptima variando el número de nodos Workers registrados en el clúster ($p = 1$, $p = 2$ y $p = 4$ workers desplegados en terminales separadas o máquinas conectadas por red local).

6 Cuestionario de Análisis Conceptual y Discusión Técnica

En el informe técnico de entrega, responda rigurosamente con base en las evidencias experimentales:

1. **Colapso por Grano Fino:** Al ejecutar con baldosas de 16×16, se generan más de 8,000 llamadas RPC sobre la red. Explique técnicamente por qué el tiempo distribuido T_{dist} se degrada frente a baldosas de 64×64, analizando el costo de serialización binaria en Slice, latencia de red TCP y cambios de contexto en el Maestro.

2. **Fracaso de la Partición Estática:** Al dividir la imagen en 2 o 4 franjas fijas (`FixedCount`), compare el tiempo de cómputo reportado por cada worker. ¿Por qué ocurre un desbalance de carga tan severo y cómo el patrón *Bag of Tasks* resuelve esta limitación?

3. **Límites del ThreadPool:** En el Experimento 2, ¿qué ocurrió al elevar el pool de hilos a 64? ¿La velocidad creció proporcionalmente o se degradó? Explique el fenómeno de contención de CPU y alternancia de contexto (*thread context switching*).

4. **Ley de Amdahl:** A partir de la curva de aceleración obtenida en el Experimento 3, estime la fracción estrictamente secuencial $(1 - f)$ del sistema impuesta por la generación de tareas, encolado y reensamblado en el Maestro.

7 Rúbrica de Evaluación Modularizada (Escala Institucional 0.0 a 5.0)

Criterio / Dimensión	Pts	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
1. Arquitectura y Ejecución Funcional	**1.8**	El clúster Master-Worker opera bajo el modelo de extracción asíncrona (*Work-Pulling*). Notificación, extracción y consolidación incremental funcionan sin bloqueos. La imagen generada es 100 % idéntica al monolito.	El clúster procesa y genera la imagen, pero presenta bloqueos ocasionales por sincronización o requiere reinicios manuales entre ejecuciones.	El clúster falla, produce imágenes corruptas o el Maestro colapsa al intentar procesar las tareas.
2. Patrón Strategy y Parametrización	**1.0**	Implementación limpia y modular del patrón Strategy. Permite alternar dinámicamente entre partición por franjas y por baldosas mediante parámetros de consola sin recompilar.	Las estrategias funcionan pero están acopladas rígidamente al código del Maestro o requieren modificar constantes en el código fuente.	No implementa el patrón Strategy o solo soporta una partición fija sin capacidad de configuración.
3. Rigor Experimental y Benchmarking	**1.2**	Ejecución completa y metódica de los tres experimentos (Granularidad, ThreadPool y Escalabilidad). Tablas de datos consistentes con cálculo exacto de Speedup (S) y Eficiencia (E).	Ejecuta los experimentos de forma incompleta, omite configuraciones clave o presenta discrepancias en el cálculo de métricas.	No presenta datos experimentales o las mediciones son inconsistentes y no demuestran pruebas reales sobre el clúster.
4. Análisis Técnico y Discusión Arquitectónica	**1.0**	Respuestas sobresalientes al cuestionario conceptual. Justifica con profundidad matemática y arquitectónica el sobrecosto de red, desbalance de carga, saturación de hilos y Ley de Amdahl.	Respuestas descriptivas pero con poco sustento en los datos experimentales o argumentación teórica superficial.	Respuestas erróneas, triviales o copia textual sin análisis propio del equipo.
CALIFICACIÓN TOTAL	**5.0**	**Escala Institucional Universidad Icesi: Aprobatoria $\geq$ 3.0 • NRC: 12378 / 12377**		

INSTRUCCIONES DE ENTREGA Y METODOLOGÍA

MODALIDAD: EQUIPOS DE 2 O 3

1. **Repositorio de Código (Git):** Alojar el código completo en el repositorio asignado en la organización de GitHub del curso (`Compu-1`), incluyendo historial de commits de los integrantes.
2. **Informe Técnico (`INFORME.md`):** Incluir tablas de resultados, gráficas de Speedup (S vs. Tamaño de Baldosa y S vs. Hilos) y respuestas al Cuestionario de Análisis Conceptual.
3. **Bitácora de Uso Ético de IAG (Nivel 3):** Declarar en una sección del informe los prompts clave utilizados, la asistencia recibida para depurar y la justificación crítica del código incorporado.